

MarketMe Technical Documentation

Technical reference for developers, operators, and contributors working on the MarketMe codebase.

For setup instructions see [README.md](#). For architecture overview see [ARCHITECTURE.md](#). For auth/data boundaries see [auth-and-data.md](#).

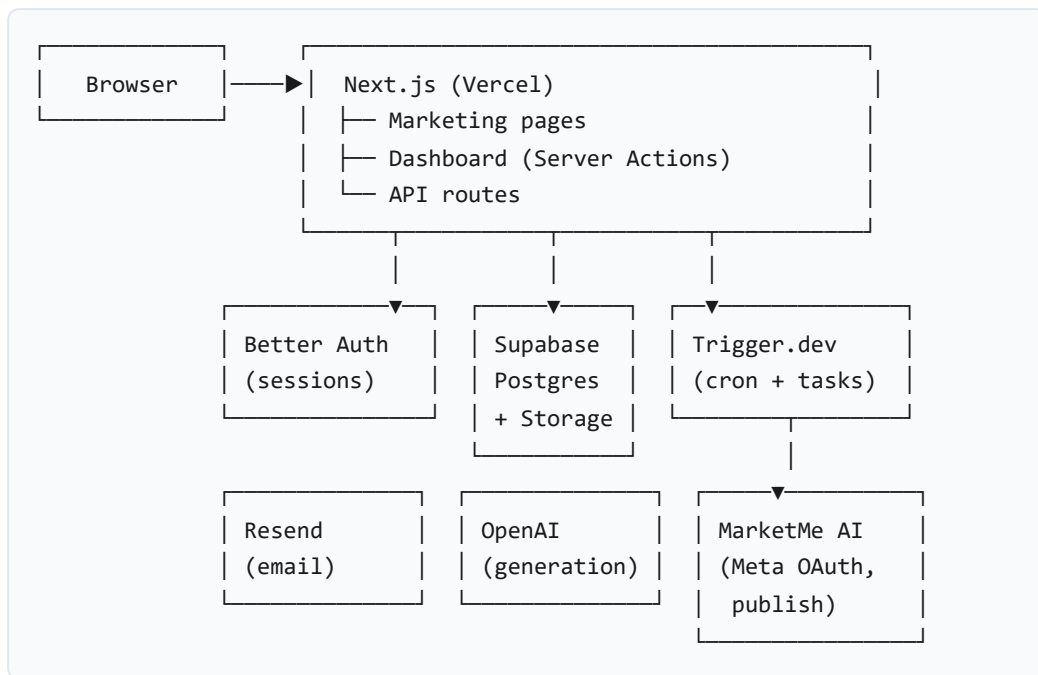
Table of contents

1. [System overview](#)
 2. [Technology stack](#)
 3. [Repository structure](#)
 4. [Authentication and authorization](#)
 5. [Data layer](#)
 6. [Application routes](#)
 7. [Server actions](#)
 8. [API routes](#)
 9. [Domain services](#)
 10. [Background jobs \(Trigger.dev\)](#)
 11. [External integrations](#)
 12. [Post lifecycle](#)
 13. [Environment variables](#)
 14. [Deployment](#)
 15. [Database schema](#)
 16. [CI/CD and quality gates](#)
 17. [Related documentation](#)
-

System overview

MarketMe is a **Next.js 16 monolith** (App Router) deployed on Vercel. It serves:

- A marketing website (static/SSR pages)
- An authenticated dashboard (client + server actions)
- REST API route handlers
- Background jobs via Trigger.dev



Critical architectural split

Concern	System
Sessions, login, OAuth	Better Auth → PostgreSQL via <code>DATABASE_URL</code>
Business data, posts, storage	Supabase → same Postgres project
User identity	Better Auth <code>user.id</code> (text) stored as <code>user_id</code> on app rows
Instagram OAuth tokens	MarketMe AI FastAPI service (not in this repo)

Better Auth handles authentication. Supabase provides Postgres, Storage, and RLS — but **not** Supabase Auth.

Technology stack

Layer	Technology	Version / notes
Framework	Next.js (App Router)	16.x
UI	React	19.x
Language	TypeScript	5.x
Styling	Tailwind CSS	4.x
Components	shadcn/ui (Radix + Base UI)	—
Animation	Framer Motion	—
Auth	Better Auth	Email/password, Google OAuth, magic link
Database	PostgreSQL via Supabase	SQL migrations in <code>supabase/migrations/</code>
Storage	Supabase Storage	<code>generated-content</code> bucket
Background jobs	Trigger.dev	v4 (<code>@trigger.dev/sdk</code>)
AI	OpenAI / OpenRouter	Captions, DALL-E images
External AI/publish	MarketMe AI API	FastAPI on Render (typical)
Email	Resend	Magic links, contact form
Billing	Stripe	Plan limits enforced in code
Stock media	Pexels API	Studio search
Canvas	Konva + react-konva	Studio editor
Hosting	Vercel	Frontend + API + daily cron
CI	GitHub Actions	typecheck, lint, build

Not used: Prisma. Schema is managed via raw SQL migrations.

Repository structure

```
marketme/
├─ app/                                # Next.js App Router
│   └─ page.tsx                        # Marketing home
│   └─ features/, pricing/, blog/, help/, ...
│   └─ login/, signup/, onboarding/
│   └─ dashboard/                      # Authenticated app
│       └─ layout.tsx                  # Auth gate + shell
│           └─ */page.tsx              # Route pages
│               └─ */actions.ts        # Server actions per feature
│   └─ api/                            # REST route handlers
├─ components/
│   └─ marketing/                      # Public site
│   └─ dashboard/                     # App UI by feature
│   └─ auth/                          # Login/signup
│   └─ ui/                            # shadcn primitives
├─ lib/
│   └─ auth.ts                        # Better Auth server config
│   └─ auth-client.ts                 # Dashboard client auth
│   └─ auth-session.ts               # Marketing navbar session
│   └─ supabase/                     # admin, server, client, middleware
│   └─ services/                     # Domain logic (~20 services)
│   └─ billing-utils.ts               # Plan definitions
├─ types/                             # Shared TypeScript types
├─ hooks/                             # Shared React hooks
├─ src/trigger/                       # Trigger.dev task definitions
├─ supabase/migrations/               # SQL schema (39+ migrations)
├─ scripts/                           # dev, env check, migrations
├─ docs/                              # Documentation
├─ proxy.ts                           # Next.js 16 request proxy (auth gates)
├─ trigger.config.ts                  # Trigger.dev project config
└─ vercel.json                        # Vercel cron config
```

Authentication and authorization

Server configuration

- **Config:** `lib/auth.ts`
- **Handler:** `app/api/auth/[...all]/route.ts`
- **Session reads:** `lib/supabase/server-auth.ts` → `getAuthenticatedUser()`

Providers

- Email/password
- Google OAuth

- Magic link (via Resend)

Route protection

`proxy.ts` (Next.js 16 request proxy, replaces middleware) gates:

- `/dashboard/*` — requires session
- `/onboarding` — requires session
- `/api/auth/*` — Better Auth handler

Unauthenticated users are redirected to `/login`.

Post-auth redirect

`lib/post-auth-redirect.ts`:

- Has business profile → `/dashboard`
- No profile → `/onboarding`

Admin access

Admin users have `is_admin = true` on `public."user"`. Admin routes:

- `/dashboard/admin` — Admin Console
- `/admin` — redirect shim

See [admin-dashboard.md](#).

Vercel OAuth notes

- Never set `BETTER_AUTH_URL=http://localhost:3000` on Vercel.
 - Register `https://<deploy-host>/api/auth/callback/google` in Google Cloud Console for each host.
-

Data layer

Access patterns

Pattern	Location	When to use
Server Action	<code>app/dashboard/*/actions.ts</code>	Dashboard mutations
Admin client	<code>getSupabaseAdmin()</code> in <code>lib/supabase/admin.ts</code>	Server reads/writes (bypasses RLS)
Server client	<code>createClient()</code> in <code>lib/supabase/server.ts</code>	Cookie-aware (rare)
Browser client	<code>createClient()</code> in <code>lib/supabase/client.ts</code>	Client-side Supabase (limited)

Always scope queries by `user_id` from `getAuthenticatedUser()` .

Migrations

Apply via:

```
npm run migrate supabase/migrations/<file>.sql
```

Schema evolves through numbered SQL files in `supabase/migrations/` . Do not use Prisma.

Application routes

Marketing (public)

Route	Description
/	Landing page
/features	Product capabilities
/pricing	Plan comparison
/about , /customers , /contact	Company pages
/blog , /blog/[slug]	Blog
/changelog	Product changelog + RSS
/help , /help/[slug]	Help center
/privacy , /terms , /cookies , /dpa , ...	Legal/compliance

Marketing pages work without backend env vars.

Auth

Route	Description
/login	Magic link, password, Google
/signup	Registration
/onboarding	Business profile wizard
/auth/callback , /auth/confirm , /auth/complete	Auth callbacks

Dashboard (authenticated)

Route	Description
<code>/dashboard</code>	Overview
<code>/dashboard/posts</code>	Post list
<code>/dashboard/calendar</code>	Planner (day/week/month)
<code>/dashboard/inbox</code>	Instagram DMs
<code>/dashboard/studio</code>	Canvas editor
<code>/dashboard/workflows</code>	Automation
<code>/dashboard/generate</code>	AI generation
<code>/dashboard/connections</code>	Social OAuth
<code>/dashboard/settings</code>	User/workspace settings
<code>/dashboard/admin</code>	Admin console
<code>/dashboard/ai</code>	Full AI assistant

Server actions

Server actions live in `app/dashboard/*/actions.ts` and are called from client components.

File	Domain
app/dashboard/actions.ts	Dashboard-level
app/dashboard/calendar/actions.ts	Planner CRUD, approve, schedule, reschedule
app/dashboard/posts/actions.ts	Posts CRUD, publish, retry, bulk delete
app/dashboard/generate/actions.ts	AI generation context and triggers
app/dashboard/studio/actions.ts	Templates, canvas save
app/dashboard/settings/actions.ts	Profile, preferences, billing
app/dashboard/account/actions.ts	Account management
app/dashboard/workflows/actions.ts	Workflow CRUD and manual runs
app/dashboard/admin/actions.ts	Admin mutations
app/onboarding/actions.ts	Profile completion
app/login/actions.ts	Login, signup, magic link

Calendar create flow (reference)

```
CreatePostModal.handleSubmit()
  → calendar/page.tsx handleCreatePost()
    → createCalendarPostAction()    // inserts draft with scheduled_at
    → scheduleCalendarPostAction()  // approve + transition to scheduled
    → void loadPosts()              // background refresh (non-blocking)
  → modal closes, toast shown
```

API routes

Route	Method	Purpose
/api/auth/[...all]	*	Better Auth handler
/api/business-profile	GET/PUT	Business profile CRUD
/api/content-plans	GET/POST	Content plan management
/api/content-plans/generate	POST	Trigger AI content generation
/api/posts/[id]/status	PATCH	Post status transitions
/api/posts/[id]/regenerate-caption	POST	Caption regeneration
/api/posts/[id]/generate-image	POST	Image generation trigger
/api/social/connect	POST	Initiate Instagram OAuth
/api/social/connections	GET/POST	Connection mirror
/api/social/insights	GET	Instagram insights
/api/inbox/conversations	GET	Inbox threads
/api/inbox/messages	GET/POST	Inbox messages
/api/pexels	GET	Stock image proxy
/api/cron/publish-scheduled	GET	Vercel cron publish (daily)
/api/analytics/collect	POST	First-party analytics
/api/contact	POST	Contact form
/api/health	GET	Health check

Cron route is protected by `CRON_SECRET`.

Domain services

Business logic lives in `lib/services/`. Key services:

Service	Responsibility
<code>creative-pipeline.service.ts</code>	AI pipeline: strategy → schedule → posts → briefs
<code>marketing-ai.service.ts</code>	MarketMe AI API client
<code>content.service.ts</code>	Post CRUD
<code>post-lifecycle.service.ts</code>	Status transitions, approve, schedule, publish
<code>scheduled-publishing.service.ts</code>	Cron publish logic, retry handling
<code>social-connections.service.ts</code>	Connection mirror and sync
<code>workflow.service.ts</code>	Workflow automation engine
<code>generation.service.ts</code>	AI generation orchestration
<code>generation-context.service.ts</code>	Context assembly for AI prompts
<code>brand-memory.service.ts</code>	Style learning from revise/approve/reject
<code>credits.service.ts</code>	AI credit ledger and deductions
<code>account.service.ts</code>	User account and subscription
<code>business.service.ts</code>	Business profile operations
<code>admin.service.ts</code>	Admin metrics and mutations
<code>inbox.service.ts</code>	Instagram inbox sync
<code>instagram-insights.service.ts</code>	Insights caching
<code>pexels.service.ts</code>	Stock image search
<code>moderation.service.ts</code>	Content moderation flags
<code>ai-preferences.service.ts</code>	User AI model preferences

Background jobs (Trigger.dev)

Config: `trigger.config.ts` — project `proj_tzdygkuaynmpopiwidtt` .

Tasks defined in `src/trigger/` :

Task ID	File	Schedule / trigger
business-analysis	content-generator.ts	On demand
marketing-strategy	content-generator.ts	On demand
generate-weekly-content	content-generator.ts	On demand
regenerate-caption	content-generator.ts	On demand
generate-creative-brief	content-generator.ts	On demand
generate-image	content-generator.ts	On demand
image-upload	content-generator.ts	Subtask
instagram-publishing	content-generator.ts	On demand
send-notification	content-generator.ts	Subtask
scheduled-publishing	content-generator.ts	Cron */15 * * * *
workflow-automation-sweep	workflow-automation.ts	Cron */15 * * * *
publish-service-keepalive	publish-keepalive.ts	Cron */10 * * * *

Scheduled publishing

Requires `ENABLE_AUTO_PUBLISH=true` (Or `INSTAGRAM_PUBLISH_ENABLED`) on both Vercel and Trigger.dev.

Publishes posts where:

- `status = 'scheduled'`
- `approved_at` is set
- `scheduled_at <= now()`

Uses tokens stored on MarketMe AI — user need not be online.

Deploy Trigger.dev

```
npx trigger.dev@latest deploy
```

Env vars are synced from Trigger.dev project settings.

External integrations

MarketMe AI API

FastAPI service (typically on Render). Env: `MARKETME_AI_API_URL` , `MARKETME_AI_API_KEY` .

Endpoints used by this app:

Endpoint	Purpose
<code>POST /api/v1/strategy/generate</code>	Marketing strategy
<code>POST /api/v1/schedules/generate</code>	Weekly schedule
<code>POST /api/v1/posts/generate</code>	Post captions
<code>POST /api/v1/creative/generate</code>	Creative briefs
<code>POST /api/v1/publish/instagram</code>	Publish to Instagram
<code>GET /api/v1/auth/meta/login</code>	Meta OAuth initiation
<code>GET /api/v1/auth/meta/callback</code>	Meta OAuth callback

Client: `lib/services/marketing-ai.service.ts`

Pipeline: `lib/services/creative-pipeline.service.ts`

Instagram OAuth flow

```
User clicks Connect
  → POST /api/social/connect (resolves business_profiles.id)
  → Redirect to MarketMe AI /api/v1/auth/meta/login?business_profile_id=...
  → Meta OAuth
  → AI API callback stores tokens
  → Redirect to frontend (rewritten by proxy.ts to /dashboard/connections)
  → POST /api/social/connections (local mirror in business_social_connections)
```

OAuth tokens live on MarketMe AI, not in this Next.js app.

Stripe

Billing integration via `STRIPE_SECRET_KEY` . Plan limits defined in `lib/billing-utils.ts` :

Plan	Workspaces	Members	Profiles	Posts/mo	Credits/mo	Price
free	1	1	1	10	50	\$0
pro	3	3	5	100	500	\$29
team	10	10	25	∞	2,000	\$79

Post lifecycle

Defined in `lib/services/post-lifecycle.service.ts` .

Status enum

`draft` | `approved` | `scheduled` | `published` | `failed` | `rejected`

Allowed transitions

```

draft    → approved, rejected
approved → scheduled, draft, rejected
scheduled → published, failed, approved
published → (terminal)
failed    → draft, scheduled
rejected  → draft

```

Key functions

Function	Behavior
<code>transitionPostStatus()</code>	Validates and applies status change
<code>approveAndSchedulePost()</code>	Approve draft → queue for publish
<code>retryFailedPost()</code>	Re-queue failed post or return to draft
<code>publishPostNow()</code>	Immediate publish via MarketMe AI

Insert defaults

`lib/insert-scheduled-post.ts` creates posts as `draft` by default with `scheduled_at` set. Scheduling requires explicit transition to `scheduled` via `approveAndSchedulePost()` .

Environment variables

Full reference: `.env.example`

Required (auth + dashboard)

```
DATABASE_URL
BETTER_AUTH_SECRET
NEXT_PUBLIC_SUPABASE_URL
NEXT_PUBLIC_SUPABASE_ANON_KEY
SUPABASE_SERVICE_ROLE_KEY
```

Auth URLs

```
BETTER_AUTH_URL
NEXT_PUBLIC_SITE_URL
GOOGLE_CLIENT_ID
GOOGLE_CLIENT_SECRET
AUTH_SESSION_EXPIRES_IN
AUTH_SESSION_UPDATE_AGE
```

Integrations

```
RESEND_API_KEY
RESEND_FROM_EMAIL
CONTACT_FROM_EMAIL
MARKETME_AI_API_URL
MARKETME_AI_API_KEY
MARKETME_AI_BUSINESS_ID
OPENAI_API_KEY
TRIGGER_SECRET_KEY
PEXELS_API_KEY
STRIPE_SECRET_KEY
```

Publishing

```
ENABLE_AUTO_PUBLISH
INSTAGRAM_PUBLISH_ENABLED
CRON_SECRET
```

Storage / analytics / admin

```
NEXT_PUBLIC_SUPABASE_BUCKET
ANALYTICS_VISITOR_SALT
ADMIN_EMAILS
```

Legal / branding

```
NEXT_PUBLIC_LEGAL_ENTITY_NAME
NEXT_PUBLIC_LEGAL_ADDRESS
NEXT_PUBLIC_COMPANY_REGISTRATION
NEXT_PUBLIC_LEGAL_EMAIL
NEXT_PUBLIC_PRIVACY_EMAIL
NEXT_PUBLIC_SUPPORT_EMAIL
```

Verify locally:

```
npm run check-env
```

Deployment

Vercel (frontend + API)

- Branch previews deploy automatically.
- Production typically tracks `main` OR `development`.
- Daily cron: `/api/cron/publish-scheduled` (see `vercel.json`).

Trigger.dev (background jobs)

Deploy separately:

```
npx trigger.dev@latest deploy
```

Ensure `ENABLE_AUTO_PUBLISH`, Supabase credentials, and `MARKETME_AI_API_URL` are set in Trigger.dev env.

Supabase

- Postgres + Storage hosted on Supabase.
- Apply migrations with `npm run migrate`.

MarketMe AI (external)

- FastAPI service on Render (typical).
- Must set `FRONTEND_URL` to match Vercel deployment host.
- Stores Meta OAuth tokens and handles publish API calls.

Deployment checklist

- ☐ `BETTER_AUTH_URL` matches deployment host (not localhost on Vercel)
- ☐ Google OAuth redirect URIs registered for each host
- ☐ `ENABLE_AUTO_PUBLISH=true` on Vercel and Trigger.dev
- ☐ Trigger.dev deployed after task changes
- ☐ Supabase migrations applied
- ☐ `MARKETME_AI_API_URL` and key configured on both Vercel and Trigger.dev

Database schema

Schema managed via SQL migrations in `supabase/migrations/` . Key tables:

Auth (Better Auth)

Table	Purpose
<code>"user"</code>	User accounts (<code>is_admin</code> flag)
<code>"session"</code>	Active sessions
<code>"account"</code>	OAuth provider links
<code>"verification"</code>	Magic link tokens

Core app

Table	Purpose
<code>business_profiles</code>	One per user; industry, tone, brand assets, style memory
<code>content_plans</code>	Weekly AI content plans
<code>posts</code>	UUID PK; status, caption, image, schedule, approval metadata
<code>studio_templates</code>	Saved canvas templates
<code>user_subscriptions</code>	Plan, Stripe IDs, credits balance
<code>user_preferences</code>	Timezone, week start
<code>user_ai_preferences</code>	AI model choices
<code>business_social_connections</code>	Local Instagram connection mirror
<code>workflows</code>	Automation definitions
<code>workflow_runs</code>	Run history
<code>credit_transactions</code>	Credit ledger
<code>generations</code>	AI generation audit
<code>moderation_flags</code>	Content moderation
<code>inbox_messages</code>	Instagram inbox
<code>instagram_account_insights</code>	Cached insights
<code>admin_audit_events</code>	Admin action log
<code>product_events</code> , <code>page_events</code> , <code>performance_events</code>	Analytics

RLS

RLS is enabled on most tables. Server code typically uses `getSupabaseAdmin()` with explicit `user_id` filtering rather than relying on RLS session context.

CI/CD and quality gates

GitHub Actions (`.github/workflows/ci.yml`) on push/PR:

1. `npm ci`
2. `npm run typecheck`
3. `npm run lint`
4. `npm run build` (placeholder env vars)

Local pre-push:

```
npm run validate # typecheck + lint
```

Next.js 16 conventions

- Use "use client" for next/dynamic with `{ ssr: false }`.
- Prefer `better-auth/client` over `better-auth/react` for client modules.
- Request proxy lives in `proxy.ts` (not `middleware.ts`).
- Read `node_modules/next/dist/docs/` for API changes — Next.js 16 has breaking changes from earlier versions.

See [AGENTS.md](#).

Related documentation

Document	Contents
README.md	Setup, scripts, troubleshooting
ARCHITECTURE.md	High-level architecture, Instagram OAuth, CI/CD
auth-and-data.md	Auth vs Supabase split, client patterns
admin-dashboard.md	Admin console
USER-GUIDE.md	End-user product guide
.env.example	Environment variable reference
.agents/rules/workflow.md	Contributor workflow

Last updated: August 2026